

ANALYTICSWITHANAND

SQL SERVER TRAINING SERIES

SQL Server Functions

A Complete Teaching Guide to Built-in Functions and User-Defined Functions
(T-SQL)

Audience Level: Intermediate (comfortable with SELECT, JOIN, GROUP BY)

Covers: Scalar · Aggregate · Window · System · User-Defined Functions

Applies to: SQL Server 2017 – 2022 / Azure SQL Database

Table of Contents

This table updates automatically. In Microsoft Word: right-click anywhere inside it and choose “Update Field” → “Update entire table.”

Table of Contents.....	1
How to Use This Guide.....	1
Learning Objectives.....	1
How the Guide Is Organized.....	1
Sample Schema Used Throughout.....	1
Module 0: The Function Landscape.....	1

Module 1: String Functions.....	1
1.1 Measuring and Extracting.....	1
1.2 Cleaning and Reshaping.....	1
1.3 Combining Values.....	1
1.4 Splitting a Delimited String into Rows.....	1
Module 2: Date and Time Functions.....	1
2.1 Getting the Current Date/Time.....	1
2.2 Adding, Subtracting, Extracting.....	1
2.3 A Real Reporting Pattern: Ageing Buckets.....	1
Module 3: Mathematical and Conversion Functions.....	1
3.1 Mathematical Functions.....	1
3.2 Conversion Functions.....	1
3.3 NULL-Handling and Logical Functions.....	1
Module 4: Aggregate Functions.....	1
4.1 The Core Five.....	1
4.2 Beyond SUM/COUNT/AVG.....	1
4.3 Subtotals with ROLLUP.....	1
Module 5: Window Functions.....	1
5.1 The OVER() Clause.....	1
5.2 Ranking Functions.....	1
5.3 “Top N per Group” — the Classic Pattern.....	1
5.4 Offset Functions: LAG and LEAD.....	1
5.5 Running Totals and Moving Averages.....	1
5.6 FIRST_VALUE and LAST_VALUE.....	1
Module 6: System and Metadata Functions.....	1
Module 7: User-Defined Functions — Scalar UDFs.....	1
7.1 Anatomy of a Scalar UDF.....	1
7.2 A More Realistic Example: Business Logic.....	1
7.3 Deterministic vs. Non-Deterministic.....	1
Module 8: User-Defined Functions — Table-Valued Functions.....	1
8.1 Inline Table-Valued Functions (iTVF).....	1
8.2 Multi-Statement Table-Valued Functions (MSTVF).....	1
8.3 iTVF vs. MSTVF — Which One and Why.....	1

Module 9: Function Performance and Best Practices.....	1
9.1 SARGability: Functions in WHERE Clauses.....	1
9.2 Scalar UDFs in Row-by-Row Contexts.....	1
9.3 Quick Decision Guide.....	1
Quick-Reference Cheat Sheet.....	1
String.....	1
Date / Time.....	1
Math / Conversion.....	1
Logical / NULL.....	1
Aggregate.....	1
Window.....	1
User-Defined.....	1
Hands-On Practice Set.....	1
Part A — Scalar Functions.....	1
Part B — Aggregates and Window Functions.....	1
Part C — User-Defined Functions.....	1
Part D — Performance.....	1
Appendix: Further Reading.....	1

How to Use This Guide

This guide is designed for a cohort-style, instructor-led SQL Server session, but it also works as a self-paced reference. It assumes you can already write SELECT, JOIN, WHERE, and GROUP BY queries comfortably — the focus here is entirely on functions: what SQL Server gives you out of the box, and how to build your own.

Learning Objectives

By the end of this guide, you will be able to:

- Classify SQL Server functions into scalar, aggregate, window, and system categories, and choose the right one for a given problem.
- Use the most common string, date/time, mathematical, conversion, and NULL-handling functions confidently.

- Write window (analytic) functions — ROW_NUMBER, RANK, LAG/LEAD, running totals — to solve ranking and trend problems without self-joins.
- Design, create, and call your own scalar and table-valued User-Defined Functions (UDFs).
- Recognize the performance traps of UDFs and non-SARGable function usage, and apply the fixes.

How the Guide Is Organized

Each module follows the same rhythm: a short concept explanation, the syntax pattern, one or more worked examples against a common sample schema, a quick-reference table, and a callout box for tips or pitfalls. Modules build on each other but can also be taught as standalone 60–90 minute sessions — a natural fit if you are slotting this into a modular 4-hour training day.

Sample Schema Used Throughout

All examples reference a small, consistent retail schema so you can follow along without re-reading table definitions every time:

```
Customers (CustomerID, FirstName, LastName, Email, City, SignupDate)
Orders (OrderID, CustomerID, OrderDate, Status, TotalAmount)
OrderItems (OrderItemID, OrderID, ProductID, Quantity, UnitPrice)
Products (ProductID, ProductName, Category, UnitCost)
```

Module 0: The Function Landscape

Before diving into individual functions, it helps to see the whole map. SQL Server functions fall into a handful of clean categories — understanding which bucket a function belongs to tells you a lot about how it behaves (one value in/out? one row or many? can it appear in WHERE?).

Category	Returns	Example	Typical Use
Scalar	Single value per row	UPPER(), GETDATE()	Transform a column value
Aggregate	Single value per group	SUM(), COUNT()	Summarize many rows into one
Window / Analytic	One value per row, computed over a window of rows	ROW_NUMBER(), LAG()	Ranking, running totals, comparisons across rows
Rowset / Table-Valued	A full result set (table)	STRING_SPLIT(), a TVF	Acts like a virtual table in FROM

System / Metadata	Server or session info	@@ROWCOUNT, USER_NAME()	Diagnostics, auditing, control flow
User-Defined (UDF)	Scalar value or table	dbo.fn_GetFullName()	Encapsulate reusable business logic

Keep this table close — every section in this guide maps back to one of these six rows.

Module 1: String Functions

String functions clean, reshape, and combine text data. They are the functions you will reach for most often when preparing data for reports or fixing messy source data.

1.1 Measuring and Extracting

```
SELECT
    FirstName,
    LEN(FirstName)           AS NameLength,
    LEFT(FirstName, 3)       AS First3,
    RIGHT(Email, 4)          AS LastFourOfEmail,
    SUBSTRING(Email, 1, CHARINDEX('@', Email) - 1) AS Username
FROM Customers;
```

Function	Syntax	Description
LEN	LEN(string)	Character count, excluding trailing spaces.
LEFT / RIGHT	LEFT(string, n)	First / last n characters.
SUBSTRING	SUBSTRING(string, start, length)	Extract a slice starting at position start (1-based).
CHARINDEX	CHARINDEX(find, string, [start])	Position of first occurrence, 0 if not found.
PATINDEX	PATINDEX('%pattern%', string)	Like CHARINDEX but supports wildcards.

1.2 Cleaning and Reshaping

```

SELECT
    UPPER(LastName)                AS LastNameUpper,
    TRIM(' ' FROM ' Mumbai ')     AS Trimmed,
    REPLACE(Email, 'gmail.com', 'outlook.com') AS SwappedDomain,
    STUFF('2026-06-20', 5, 1, '/') AS PartialReplace,
    REPLICATE('*', 5)              AS Stars,
    REVERSE('AnalyticsWithAnand')  AS Reversed
FROM Customers;

```

Function	Syntax	Description
UPPER / LOWER	UPPER(string)	Case conversion.
TRIM	TRIM(string)	Removes leading/trailing spaces (SQL Server 2017+).
REPLACE	REPLACE(string, old, new)	Replaces all occurrences of old with new.
STUFF	STUFF(string, start, length, new)	Deletes length characters from start, then inserts new.
REPLICATE	REPLICATE(string, n)	Repeats a string n times.
REVERSE	REVERSE(string)	Reverses character order.

1.3 Combining Values

```

-- CONCAT ignores NULLs (safer than using +)
SELECT CONCAT(FirstName, ' ', LastName, ' (', City, ')') AS Label
FROM Customers;

-- CONCAT_WS: one separator, many values, skips NULLs automatically
SELECT CONCAT_WS(' ', City, 'India') AS Location
FROM Customers;

-- STRING_AGG: roll up many rows into one delimited string (aggregate function)
SELECT
    o.CustomerID,
    STRING_AGG(p.ProductName, ' ') WITHIN GROUP (ORDER BY p.ProductName) AS
ProductsOrdered
FROM Orders o
JOIN OrderItems oi ON oi.OrderID = o.OrderID
JOIN Products p ON p.ProductID = oi.ProductID
GROUP BY o.CustomerID;

```

Common Pitfall

Plain string concatenation with + returns NULL the moment any input is NULL: 'Hello' + NULL = NULL. Use CONCAT() instead — it treats NULL as an empty string and is almost always what you actually want.

1.4 Splitting a Delimited String into Rows

STRING_SPLIT() is a rowset function — it returns a table, not a scalar, so it belongs in FROM, not SELECT:

```
SELECT value AS Tag
FROM STRING_SPLIT('analytics,sql,snowflake,matillion', ',');

-- SQL Server 2022+: request the position with enable_ordinal
SELECT value, ordinal
FROM STRING_SPLIT('analytics,sql,snowflake,matillion', ',', 1);
```

Try It Yourself

Write a query that returns each customer's full name, their email domain (the part after @), and a single comma-separated list of cities for all customers sharing that domain.

Module 2: Date and Time Functions

Date logic is where a surprising amount of reporting code lives — fiscal periods, ageing buckets, “last 30 days” filters. SQL Server's date functions all revolve around a small set of “datepart” keywords (year, quarter, month, day, week, hour, etc.) that you pass as the first argument.

2.1 Getting the Current Date/Time

Function	Returns	Notes
GETDATE()	datetime, server local time	Most common; precision to ~3ms.
SYSDATETIME()	datetime2, server local time	Higher precision (100ns).
GETUTCDATE()	datetime, UTC	Use for storage when servers span time zones.
SYSDATETIMEOFFSET()	datetimeoffset	Includes the UTC offset.

2.2 Adding, Subtracting, Extracting

```

SELECT
    OrderDate,
    DATEADD(DAY, 30, OrderDate)           AS PlusThirtyDays,
    DATEDIFF(DAY, OrderDate, GETDATE())   AS DaysSinceOrder,
    DATEPART(QUARTER, OrderDate)          AS OrderQuarter,
    DATENAME(WEEKDAY, OrderDate)          AS OrderDayName,
    YEAR(OrderDate)                       AS OrderYear,
    EOMONTH(OrderDate)                    AS LastDayOfOrderMonth
FROM Orders;

```

Function	Syntax	Description
DATEADD	DATEADD(datepart, n, date)	Adds n units of datepart to date (n can be negative).
DATEDIFF	DATEDIFF(datepart, start, end)	Counts datepart boundaries crossed between two dates.
DATEPART	DATEPART(datepart, date)	Returns the requested part as an integer.
DATENAME	DATENAME(datepart, date)	Returns the requested part as a string (e.g. 'June').
EOMONTH	EOMONTH(date, [months])	Last calendar day of the month, optionally offset by months.
DATEFROMPARTS	DATEFROMPARTS(y, m, d)	Builds a date from integer parts — safer than string concatenation.

Common Pitfall

DATEDIFF(YEAR, '2025-12-31', '2026-01-01') returns 1, even though only one day passed — DATEDIFF counts how many datepart boundaries were crossed, not elapsed duration. For true age-in-years logic, combine DATEDIFF with a DATEADD comparison.

2.3 A Real Reporting Pattern: Ageing Buckets

```

SELECT
    o.OrderID,
    o.OrderDate,
    DATEDIFF(DAY, o.OrderDate, GETDATE()) AS AgeInDays,
    CASE
        WHEN DATEDIFF(DAY, o.OrderDate, GETDATE()) <= 30 THEN '0-30 days'
        WHEN DATEDIFF(DAY, o.OrderDate, GETDATE()) <= 90 THEN '31-90 days'
        ELSE '90+ days'
    END AS AgeBucket
FROM Orders o
WHERE o.Status = 'Open';

```

Try It Yourself

Write a query that returns, for each calendar month in the Orders table, the month name and the total order amount for that month, ordered chronologically (hint: DATENAME and YEAR/MONTH together).

Module 3: Mathematical and Conversion Functions

3.1 Mathematical Functions

```
SELECT
    UnitPrice,
    ROUND(UnitPrice, 1) AS Rounded1dp,
    CEILING(UnitPrice) AS RoundedUp,
    FLOOR(UnitPrice) AS RoundedDown,
    ABS(-UnitPrice) AS AbsoluteValue,
    POWER(Quantity, 2) AS QuantitySquared,
    SQRT(Quantity) AS QuantitySqrt
FROM OrderItems;
```

Function	Syntax	Description
ROUND	ROUND(num, length)	Rounds to length decimal places (negative length rounds left of the decimal).
CEILING / FLOOR	CEILING(num)	Rounds up / down to the nearest integer.
ABS	ABS(num)	Absolute (non-negative) value.
POWER	POWER(base, exp)	base raised to exp.
SQRT / SQUARE	SQRT(num)	Square root / square.
RAND	RAND([seed])	Pseudo-random float between 0 and 1.

3.2 Conversion Functions

Converting between types is unavoidable — dates from strings, numbers from text imports, decimals for currency display. SQL Server gives you two families: the ones that throw an error on failure (CAST, CONVERT) and the safer TRY_ variants that return NULL instead.

```

-- CAST: ANSI-standard, simplest syntax
SELECT CAST(TotalAmount AS INT) AS RoundedAmount
FROM Orders;

-- CONVERT: T-SQL specific, supports a numeric "style" code (very useful for dates)
SELECT CONVERT(VARCHAR(10), OrderDate, 103) AS UK_Format -- dd/mm/yyyy
FROM Orders;

-- TRY_CAST / TRY_CONVERT: return NULL on failure instead of erroring out the whole batch
SELECT TRY_CAST('20-June-2026' AS DATE) AS ParsedOrNull;

-- FORMAT: most flexible, but slower — best for final report-layer formatting only
SELECT FORMAT(TotalAmount, 'C', 'en-IN') AS FormattedCurrency
FROM Orders;

```

Function	Throws on Failure?	Best For
CAST(expr AS type)	Yes	Simple, portable type changes.
CONVERT(type, expr, style)	Yes	Date/number formatting with style codes.
TRY_CAST / TRY_CONVERT	No — returns NULL	Cleaning untrusted/imported data.
PARSE / TRY_PARSE	Yes / No	Culture-aware parsing (e.g. European decimals).
FORMAT(value, fmt, culture)	No — returns error string	Final display formatting; avoid in large queries (slow).

Common Pitfall

FORMAT() is the most readable conversion function but also the slowest, because it calls into the .NET CLR for every row. Reach for it on the final reporting layer (small result sets), never on millions of rows in a transactional query.

3.3 NULL-Handling and Logical Functions

```

SELECT
    CustomerID,
    ISNULL(MiddleName, "") AS MiddleNameSafe,
    COALESCE(MobilePhone, HomePhone, 'No phone on file') AS BestContactNumber,
    NULLIF(DiscountCode, "") AS DiscountCodeOrNull,
    IIF(TotalAmount > 5000, 'High Value', 'Standard') AS OrderTier,
    CHOOSE(DATEPART(QUARTER, OrderDate), 'Q1','Q2','Q3','Q4') AS QuarterLabel
FROM Orders;

```

Function	Syntax	Description
ISNULL	ISNULL(expr, replacement)	2 arguments only; replacement type must match expr's type.
COALESCE	COALESCE(e1, e2, ...)	ANSI standard; returns the first non-NULL of any number of expressions.
NULLIF	NULLIF(e1, e2)	Returns NULL if e1 = e2, otherwise returns e1. Great for avoiding divide-by-zero.
IIF	IIF(condition, true, false)	Shorthand for a simple two-branch CASE (SQL Server 2012+).
CHOOSE	CHOOSE(index, v1, v2, ...)	1-based positional lookup.

Common Pitfall

ISNULL vs. COALESCE look interchangeable but aren't: ISNULL takes exactly two arguments and returns a type based on the first argument (which can silently truncate), while COALESCE follows standard SQL type-precedence rules across all arguments. Default to COALESCE unless you have a specific reason not to — it's also portable to other databases.

Try It Yourself

Rewrite a query that uses ISNULL(Discount, 0) / NULLIF(Quantity, 0) to safely compute a per-unit discount percentage, avoiding both NULL propagation and divide-by-zero errors.

Module 4: Aggregate Functions

Aggregate functions collapse many rows into one value per group. They are the functions behind every GROUP BY query, and the foundation you need solid before window functions — which reuse the same function names in a very different way.

4.1 The Core Five

```

SELECT
    p.Category,
    COUNT(*)                AS OrderLineCount,
    COUNT(DISTINCT oi.OrderID) AS DistinctOrders,
    SUM(oi.Quantity * oi.UnitPrice) AS Revenue,
    AVG(oi.UnitPrice)        AS AvgUnitPrice,
    MIN(oi.UnitPrice)        AS CheapestLine,
    MAX(oi.UnitPrice)        AS MostExpensiveLine
FROM OrderItems oi
JOIN Products p ON p.ProductID = oi.ProductID
GROUP BY p.Category
HAVING SUM(oi.Quantity * oi.UnitPrice) > 10000
ORDER BY Revenue DESC;

```

Common Pitfall

WHERE filters rows before grouping; HAVING filters groups after aggregation. A frequent beginner mistake is trying to use an aggregate function inside WHERE — SQL Server will reject it. If the filter depends on SUM(), COUNT(), etc., it belongs in HAVING.

4.2 Beyond SUM/COUNT/AVG

Function	Syntax	Description
STRING_AGG	STRING_AGG(expr, sep) WITHIN GROUP (ORDER BY ...)	Concatenates group values into one delimited string (2017+).
STDEV / STDEVP	STDEV(expr)	Sample / population standard deviation.
VAR / VARP	VAR(expr)	Sample / population variance.
GROUPING	GROUPING(col)	Flags whether a row is a subtotal produced by ROLLUP/CUBE.
APPROX_COUNT_DISTINCT	APPROX_COUNT_DISTINCT(col)	Fast, approximate distinct count for very large tables (2019+).

4.3 Subtotals with ROLLUP

GROUP BY ... WITH ROLLUP adds subtotal and grand-total rows in a single pass — useful for management-style reports without a UNION ALL hack:

```
SELECT
    p.Category,
    CONVERT(VARCHAR(7), o.OrderDate, 120) AS OrderMonth,
    SUM(oi.Quantity * oi.UnitPrice) AS Revenue,
    GROUPING(p.Category) AS IsCategorySubtotal
FROM Orders o
JOIN OrderItems oi ON oi.OrderID = o.OrderID
JOIN Products p ON p.ProductID = oi.ProductID
GROUP BY p.Category, CONVERT(VARCHAR(7), o.OrderDate, 120)
WITH ROLLUP;
```

Try It Yourself

Write a query that returns each customer's order count, total spend, and average order value — but only for customers who have placed 3 or more orders.

Module 5: Window Functions

Window functions are the single highest-leverage topic in this guide. They compute a value for each row using a “window” of related rows — without collapsing rows the way GROUP BY does. If you've ever written a self-join just to rank or compare rows, a window function almost certainly replaces it with cleaner, faster SQL.

5.1 The OVER() Clause

Every window function shares the same anatomy:

```
<function>() OVER (
    [PARTITION BY column_list] -- optional: reset the window per group
    [ORDER BY column_list]      -- required for ranking/offset functions
    [ROWS|RANGE BETWEEN ...]   -- optional: frame size for aggregates
)
```

PARTITION BY is the window-function equivalent of GROUP BY — except rows are not collapsed; you still get one output row per input row.

5.2 Ranking Functions

```

SELECT
    CustomerID,
    OrderID,
    TotalAmount,
    ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS
RowNum,
    RANK()          OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS Rnk,
    DENSE_RANK() OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS
DenseRnk,
    NTILE(4)        OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS
Quartile
FROM Orders;

```

Function	Ties Behave As	Typical Use
ROW_NUMBER()	Always unique, arbitrary tiebreak	“Top N per group” queries; deduping.
RANK()	Ties share rank; gaps appear after ties	Leaderboards where gaps are meaningful.
DENSE_RANK()	Ties share rank; no gaps	Leaderboards where gaps should be hidden.
NTILE(n)	Splits rows into n roughly-equal buckets	Quartiles, deciles, percentile bucketing.

5.3 “Top N per Group” — the Classic Pattern

This is the pattern that makes ROW_NUMBER() indispensable: finding each customer's single most expensive order without a correlated subquery.

```

WITH RankedOrders AS (
    SELECT
        CustomerID,
        OrderID,
        TotalAmount,
        ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS rn
    FROM Orders
)
SELECT CustomerID, OrderID, TotalAmount
FROM RankedOrders
WHERE rn = 1;

```

Common Pitfall

Window functions cannot be referenced directly in WHERE — SQL Server evaluates WHERE before the SELECT list. Wrap the query in a CTE (as above) or a subquery, then filter on the outer layer.

5.4 Offset Functions: LAG and LEAD

LAG looks backward, LEAD looks forward — both let you compare a row to a neighboring row without a self-join.

```
SELECT
    CustomerID,
    OrderDate,
    TotalAmount,
    LAG(TotalAmount, 1) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS
PreviousOrderAmount,
    TotalAmount - LAG(TotalAmount, 1) OVER (PARTITION BY CustomerID ORDER BY OrderDate)
AS ChangeFromLastOrder,
    LEAD(OrderDate, 1) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS
NextOrderDate
FROM Orders;
```

5.5 Running Totals and Moving Averages

Aggregate functions become window functions the moment you add OVER(). The frame clause (ROWS BETWEEN ...) controls exactly which rows participate.

```
SELECT
    CustomerID,
    OrderDate,
    TotalAmount,
    SUM(TotalAmount) OVER (
    PARTITION BY CustomerID
    ORDER BY OrderDate
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal,
    AVG(TotalAmount) OVER (
    PARTITION BY CustomerID
    ORDER BY OrderDate
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Trailing3OrderAvg
FROM Orders;
```

Frame Clause	Meaning
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	From the start of the partition through the current row — classic running total.
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	Current row plus the 2 immediately before it — a 3-row moving average.
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING	Current row through the end of the partition — a reverse running total.

5.6 FIRST_VALUE and LAST_VALUE

```
SELECT
    CustomerID,
    OrderDate,
    TotalAmount,
    FIRST_VALUE(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS
FirstOrderDate,
    LAST_VALUE(OrderDate) OVER (
    PARTITION BY CustomerID ORDER BY OrderDate
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS LastOrderDate
FROM Orders;
```

Common Pitfall

LAST_VALUE with the default frame only sees rows up to the current row, so it usually just returns the current row's own value — not the true last row of the partition. You almost always need to override the frame explicitly to ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING, as shown above.

Try It Yourself

For each product, write a query that ranks orders by revenue (using DENSE_RANK), and adds a column showing the percentage change in revenue versus the previous order for that product (using LAG).

Module 6: System and Metadata Functions

These functions answer questions about the server, the session, or the statement that just ran — essential for error handling, auditing, and control-of-flow logic in stored procedures.

Function	Returns
@@ROWCOUNT	Number of rows affected by the last statement.
SCOPE_IDENTITY()	Last identity value generated in the current scope (preferred over @@IDENTITY).
@@IDENTITY	Last identity value generated in the current session — across any scope, including triggers (avoid; use SCOPE_IDENTITY).
ERROR_NUMBER() / ERROR_MESSAGE()	Details of the error caught inside a TRY...CATCH block.
USER_NAME() / SUSER_NAME()	Current database user / server login.

DB_NAME() / OBJECT_NAME(id)	Current database name / resolves an object_id to its name.
@@SERVERNAME / @@VERSION	Server identity / SQL Server build and edition info.
<pre> BEGIN TRY INSERT INTO Orders (CustomerID, OrderDate, Status, TotalAmount) VALUES (101, GETDATE(), 'Open', 2499.00); PRINT 'Rows inserted: ' + CAST(@@ROWCOUNT AS VARCHAR(10)); PRINT 'New OrderID: ' + CAST(SCOPE_IDENTITY() AS VARCHAR(20)); END TRY BEGIN CATCH PRINT 'Error ' + CAST(ERROR_NUMBER() AS VARCHAR(10)) + ': ' + ERROR_MESSAGE(); END CATCH;</pre>	

Common Pitfall

@@ROWCOUNT resets after every statement, including the SET or PRINT statements you might add for debugging. Capture it into a variable immediately: SET @RowsAffected = @@ROWCOUNT; — don't trust it a few lines later.

Module 7: User-Defined Functions — Scalar UDFs

Everything so far has been built-in. A User-Defined Function (UDF) lets you wrap your own reusable logic — a business rule, a calculation, a formatting convention — behind a name you call just like a built-in function.

7.1 Anatomy of a Scalar UDF

A scalar UDF takes zero or more parameters and returns a single value, exactly like LEN() or ROUND() do.

```

CREATE FUNCTION dbo.fn_GetFullName
(
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50)
)
RETURNS NVARCHAR(101)
AS
BEGIN
    RETURN @FirstName + N' ' + @LastName;
END;
GO

-- Calling it: just like any built-in function
SELECT dbo.fn_GetFullName(FirstName, LastName) AS FullName
FROM Customers;

```

7.2 A More Realistic Example: Business Logic

UDFs earn their keep when they encapsulate a rule that appears in many queries — if the rule ever changes, you fix it in one place.

```

CREATE FUNCTION dbo.fn_CustomerTier (@LifetimeSpend DECIMAL(18,2))
RETURNS VARCHAR(20)
WITH SCHEMABINDING
AS
BEGIN
    RETURN CASE
        WHEN @LifetimeSpend >= 100000 THEN 'Platinum'
        WHEN @LifetimeSpend >= 50000 THEN 'Gold'
        WHEN @LifetimeSpend >= 10000 THEN 'Silver'
        ELSE 'Standard'
    END;
END;
GO

SELECT
    CustomerID,
    SUM(TotalAmount) AS LifetimeSpend,
    dbo.fn_CustomerTier(SUM(TotalAmount)) AS Tier
FROM Orders
GROUP BY CustomerID;

```

7.3 Deterministic vs. Non-Deterministic

A function is deterministic if it always returns the same output for the same input (e.g. `fn_CustomerTier` above). It is non-deterministic if it depends on something external — `GETDATE()`, `RAND()`, or a table lookup. This matters because deterministic, schema-bound scalar functions are exactly the ones SQL Server 2019+ can inline automatically (see Module 9), and the only ones you can index on in a computed column.

Concept	Meaning
WITH SCHEMABINDING	Locks the underlying objects' schema so they can't be altered/dropped without first altering the function. Required for indexed computed columns and a precondition for scalar UDF inlining.
Deterministic	Same inputs → same output, always. No calls to GETDATE(), RAND(), or non-deterministic system functions inside.

Try It Yourself

Write a scalar UDF `dbo.fn_DaysUntilDelivery(@OrderDate DATE)` that returns the number of calendar days between the order date and today, then use it in a SELECT against Orders.

Module 8: User-Defined Functions — Table-Valued Functions

A Table-Valued Function (TVF) returns a result set instead of a single value, and is used in FROM exactly like a table or a view. SQL Server gives you two flavors with very different performance characteristics — the difference matters enough that it gets its own module.

8.1 Inline Table-Valued Functions (iTVF)

An inline TVF is a single RETURN (SELECT ...) statement — no BEGIN/END block, no internal variable. SQL Server expands it into the calling query at compile time, just like a parameterized view.

```
CREATE FUNCTION dbo.fn_OrdersByCustomer (@CustomerID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT OrderID, OrderDate, Status, TotalAmount
    FROM Orders
    WHERE CustomerID = @CustomerID
);
GO

-- Calling it: use it in FROM, with CROSS/OUTER APPLY for per-row parameters
SELECT c.CustomerID, c.FirstName, o.OrderID, o.TotalAmount
FROM Customers c
CROSS APPLY dbo.fn_OrdersByCustomer(c.CustomerID) o;
```

8.2 Multi-Statement Table-Valued Functions (MSTVF)

A multi-statement TVF declares a table variable, populates it with one or more statements inside BEGIN/END, then returns it. This gives you procedural flexibility — loops, multiple INSERTs, temp logic — at a real performance cost (see 8.3).

```
CREATE FUNCTION dbo.fn_CustomerOrderSummary (@CustomerID INT)
RETURNS @Summary TABLE
(
    OrderCount INT,
    TotalSpent DECIMAL(18,2),
    FirstOrder DATE,
    LastOrder DATE
)
AS
BEGIN
    INSERT INTO @Summary (OrderCount, TotalSpent, FirstOrder, LastOrder)
    SELECT
        COUNT(*),
        SUM(TotalAmount),
        MIN(OrderDate),
        MAX(OrderDate)
    FROM Orders
    WHERE CustomerID = @CustomerID;

    RETURN;
END;
GO

SELECT * FROM dbo.fn_CustomerOrderSummary(101);
```

8.3 iTVF vs. MSTVF — Which One and Why

	Inline TVF (iTVF)	Multi-Statement TVF (MSTVF)
Body	Single RETURN (SELECT ...)	BEGIN...END block with a table variable
Optimizer treatment	Expanded into the query plan like a view — fully optimizable	Treated as a black box; historically estimated at a fixed low row count
Cardinality estimation	Accurate — same as the underlying query	Often wildly wrong — a common cause of bad join plans on large data
Procedural logic (loops, multiple statements)	Not possible	Supported
Recommendation	Default choice whenever the logic is a single query	Only when you genuinely need procedural steps; keep result sets small

Common Pitfall

MSTVFs are a frequent silent performance killer. Because the optimizer can't see inside the BEGIN/END block, older SQL Server versions assume the function returns exactly 1 row regardless of reality — and even on 2019+ with interleaved execution improvements, large MSTVFs used inside joins remain a common cause of bad query plans. If a function's body is a single SELECT, write it as an iTVF, not an MSTVF.

Try It Yourself

Convert `dbo.fn_CustomerOrderSummary` above into an equivalent inline TVF that returns the same four columns using a single RETURN (SELECT ...) statement.

Module 9: Function Performance and Best Practices

This is the module that separates a working query from a production-ready one. Functions are convenient, but convenience has a cost — and that cost is easy to miss until a table grows from 10,000 rows to 10 million.

9.1 SARGability: Functions in WHERE Clauses

A predicate is “SARGable” (Search ARGument-able) if SQL Server can use an index to seek directly to matching rows. Wrapping an indexed column in a function almost always breaks that:

```
-- NOT SARGable: the optimizer must compute YEAR(OrderDate) for every row
-- before it can compare — an index on OrderDate cannot be sought, only scanned.
SELECT * FROM Orders WHERE YEAR(OrderDate) = 2026;

-- SARGable: OrderDate itself is compared directly, so an index seek is possible.
SELECT * FROM Orders
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01';
```

Common Pitfall

This applies to ANY function on the indexed side of a comparison — `UPPER(LastName) = 'JHA'`, `CONVERT(VARCHAR, OrderDate, 103) = '20/06/2026'`, `ISNULL(Status, 'Open') = 'Open'` are all non-SARGable. The fix is almost always to rewrite the comparison so the column stays bare, and move the transformation to the literal side instead.

9.2 Scalar UDFs in Row-by-Row Contexts

Before SQL Server 2019, every call to a scalar UDF inside a SELECT against many rows executed once per row, outside the normal query engine — informally called RBAR (“Row-By-agonizing-Row”). A scalar UDF that itself queries a table, called against a million-row SELECT, can turn a sub-second query into a multi-minute one.

SQL Server Version	Scalar UDF Behavior
2017 and earlier	Always executed row-by-row; no automatic inlining.
2019+ (compat level 150+)	Scalar UDF Inlining: simple, deterministic functions can be automatically rewritten into the calling query — but only when written in a single RETURN expression and meeting several restrictions (no error handling, no table access in some cases, etc.).

Practical guidance for an intermediate developer: don't assume inlining will save you. Test with the actual execution plan (look for “Scalar UDF Inlining” in the plan XML), and for anything performance-sensitive, prefer an inline TVF with CROSS APPLY over a scalar UDF that hits a table.

9.3 Quick Decision Guide

If you need to...	Reach for...
Transform one column's value per row	Built-in scalar function, or a deterministic scalar UDF (small calculations only)
Reuse a single-query business rule across many places	Inline TVF (iTVF) + CROSS/OUTER APPLY
Run procedural logic (loops, multiple inserts) per call	Multi-statement TVF — but profile it; consider a stored procedure instead
Rank, compare to neighboring rows, or compute running totals	Window function with OVER()
Filter on a derived value from an indexed column	Rewrite the predicate so the column is bare (SARGable form)

Quick-Reference Cheat Sheet

One page, every function family from this guide, for printing or pinning above your desk.

String

Function	Purpose
LEN, LEFT, RIGHT, SUBSTRING	Measure / extract

CHARINDEX, PATINDEX	Locate a substring or pattern
UPPER, LOWER, TRIM, REPLACE, STUFF	Clean / reshape
CONCAT, CONCAT_WS, STRING_AGG	Combine values (NULL-safe)
STRING_SPLIT	Split a delimited string into rows

Date / Time

Function	Purpose
GETDATE, SYSDATETIME, GETUTCDATE	Current date/time
DATEADD, DATEDIFF	Date arithmetic
DATEPART, DATENAME, YEAR/MONTH/DAY	Extract a component
EOMONTH, DATEFROMPARTS	Build / derive dates safely

Math / Conversion

Function	Purpose
ROUND, CEILING, FLOOR, ABS, POWER, SQRT	Numeric calculations
CAST, CONVERT	Type conversion (throws on failure)
TRY_CAST, TRY_CONVERT, TRY_PARSE	Type conversion (NULL on failure)
FORMAT	Culture-aware display formatting (use sparingly)

Logical / NULL

Function	Purpose
ISNULL, COALESCE	Substitute a value for NULL (prefer COALESCE)
NULLIF	Turn a matching value into NULL (avoid divide-by-zero)
IIF, CHOOSE	Inline conditional shorthand

Aggregate

Function	Purpose
COUNT, SUM, AVG, MIN, MAX	Core summarization

STRING_AGG	Concatenate group values
STDEV, VAR	Statistical spread
GROUPING / WITH ROLLUP	Subtotal and grand-total rows

Window

Function	Purpose
ROW_NUMBER, RANK, DENSE_RANK, NTILE	Ranking within a partition
LAG, LEAD	Compare to a neighboring row
FIRST_VALUE, LAST_VALUE	Edge values of the window frame
SUM/AVG(...) OVER(...)	Running totals, moving averages

User-Defined

Type	When to Use
Scalar UDF	Small, deterministic, single-value calculations
Inline TVF (iTVF)	Default choice — single-query reusable logic
Multi-statement TVF	Only when procedural steps are unavoidable

Hands-On Practice Set

Use the sample schema from the introduction. Work through these roughly in order — each one reinforces a module above. Suggested time: 45–60 minutes for the full set in a cohort session.

Part A — Scalar Functions

1. Return each customer's full name in the format “LASTNAME, Firstname” using string functions.
2. Return each order's age in days, and a text bucket (“This week” / “This month” / “Older”) using DATEDIFF and CASE.
3. Return each order's TotalAmount rounded to the nearest 100, using ROUND with a negative length argument.
4. Build a single column that safely displays a customer's phone number, falling back through MobilePhone → HomePhone → the text 'Not provided', using COALESCE.

Part B — Aggregates and Window Functions

5. For each Category, return total revenue and the number of distinct products sold.
6. Rank each customer's orders from highest to lowest TotalAmount using DENSE_RANK, and return only rank 1 and 2 per customer.
7. Compute a running total of revenue by OrderDate across the whole Orders table (no PARTITION BY).
8. For each customer, compute the change in order amount from their previous order using LAG, and flag rows where the change exceeds 50%.

Part C — User-Defined Functions

9. Create a scalar UDF dbo.fn_OrderTier(@TotalAmount DECIMAL(18,2)) returning 'Small' / 'Medium' / 'Large' and use it in a SELECT.
10. Create an inline TVF dbo.fn_TopProductsByCategory(@Category VARCHAR(50), @TopN INT) and call it with CROSS APPLY for every distinct category.
11. Take the MSTVF dbo.fn_CustomerOrderSummary from Module 8 and rewrite it as an iTVF. Compare the two execution plans — what's different?

Part D — Performance

12. Find a query in your own codebase (or write one) that filters on YEAR(SomeDateColumn) = ... and rewrite it in SARGable form.
13. Take any scalar UDF you wrote above and check its execution plan for “Scalar UDF Inlining.” Explain in one sentence why it was or wasn't inlined.

Appendix: Further Reading

This guide covers the functions you'll use in the large majority of real T-SQL work. For deeper or edge-case reference, Microsoft's official documentation is the authoritative source and is kept current with every SQL Server release:

- Built-in Functions (Transact-SQL) — the full categorized index of every function SQL Server ships.
- CREATE FUNCTION (Transact-SQL) — full syntax reference for scalar and table-valued UDFs, including SCHEMABINDING and inlining eligibility rules.
- Scalar UDF Inlining — the specific restrictions a function must meet to be eligible for automatic inlining on SQL Server 2019+.

- Query Tuning and Optimization — background on SARGability, execution plans, and how the optimizer treats different function types.

Prepared as part of the ANALYTICSWITHANAND SQL Server training series.